Extra Add-On to My P vs NP Paper

Amine Belachhab - Independent HS researcher

In addition to the core arguments I gave in my main P vs NP paper, I now propose a specific algorithm that handles a real-world NP-complete-like problem — the Journal's crossword puzzle. While it does not settle the P vs NP question alone, it helps reinforce the argument that **not all NP problems are practically solvable by brute force unless we add structure or strategic tricks** — hence giving more evidence for **P** $\neq$ **NP**.

The Crossword Puzzle Problem

Let this be the NP-C problem to tackle:

We have a Journal-style crossword on an $n \times n$ grid, with word boxes placed both vertically and horizontally. Each word box contains a certain number of empty letter slots. We have a dictionary (set of available words). The task is to fill the grid with these words in a way that all crossing constraints are satisfied (i.e., intersecting letters match).

ALGO 1: Core Backtracking with Memory

Let the following algorithm be **ALGO 1**:

- Look at crossing word boxes and their number of letters.
- Look at the crossing letters.
- Search for words that are of the same length and have the crossing letter in the same order as needed.
- Put the word and repeat until it's no more possible.
- Then stock the sequence that led to the error to not repeat it again.
- Delete the last modification prior to the error.
- Repeat the algorithm.
- If it's impossible again to continue, delete the one before and keep going if necessary.

This algorithm is essentially a smart backtracking algorithm with pruning: it avoids repeating bad states by recording them.

ALGO 2: Full Solver on the Grid

We now extend the solver with more global planning, in what we may call **ALGO 2**:

- Look at all the words and the letter count of each one.
- Organize them according to their length.
- Look at all the word boxes and the number of empty letter places.
- Put one category of words that have the same length at the same time.
  - If impossible because these words cross, use ALGO 1 at the same time in all branches.
- Repeat and if the algorithm encounters an even or odd (except 1) number of problems relating to the beginning of the algorithm, then change the order of put words by shifting it by 1.
- If the number of problems is 1, then freeze that branch and wait for another problem to pop out, then do the shifting.
- But if the problems relate to a number of words with the same length and same placement of the crossing letters, then just swap by again shifting by one.
- And stock wrong sequences and their branches to not repeat the same mistakes!

This is basically ALGO 1 plus dynamic rearrangement and branch control. It resembles a recursive, self-correcting crossword engine that adapts on-the-go.

What This Means for P vs NP

This algorithm shows that structured NP-C problems can be made more manageable by using **heuristics, constraint propagation, and intelligent pruning**. But even with all this, we still may hit branches that require backtracking. So:

- It doesn't solve all NP-C problems in polynomial time.
- It **proves that even when a problem is solvable in practice, the general case remains hard**, unless more structural shortcuts are added.

Hence, my algorithm strengthens the idea that $\mathbf{P} \neq \mathbf{NP}$, because we still need exponential backtracking **with optimizations**, and without those, we fall back into infeasibility.

This is one more step into crushing the P = NP dream (if it ever existed).